

Sistema de Gerenciamento de Pedidos em Java

Autores: Rafael Pinto Germano Pereira
Henrique Estrela Santos
Matheus Kaick Rocha da Fonseca
Luan Vinicius Miranda da Silva
Arthur de Aquino Anjos

Agenda

O projeto teve como foco aplicar o padrão **Singleton** em Java para centralizar os dados do sistema **FoodDelivery**, garantindo organização, segurança e consistência das informações. A implementação trouxe melhorias no código, como o uso de **synchronized** no método **getInstance()**, além de consolidar o aprendizado sobre padrões de projeto e boas práticas de desenvolvimento.

1. Item 1: Fundamentação teórica

4. Item: Resultado e discussões

2. Item: Diagrama

5. Item: Considerações finais

3. Item: Metodologia

Fundamentação teórica



- O padrão Singleton garante que uma classe tenha apenas uma única instância em todo o sistema, sendo ideal quando é necessário um ponto central de controle, como um gerenciador de configurações ou uma central de dados.
- Para isso, o construtor da classe é definido como **privado**, impedindo que outras partes do código criem novas instâncias, e é criado um **método estático público**, chamado `getInstance()`, que controla o acesso ao objeto. Na primeira vez que o método é chamado, ele cria a instância; nas próximas, retorna a mesma já existente.
- No projeto *FoodDelivery*, a classe **Repository** utiliza o padrão Singleton para garantir que todas as telas do sistema acessem as mesmas informações, evitando problemas de inconsistência e facilitando o gerenciamento dos dados.

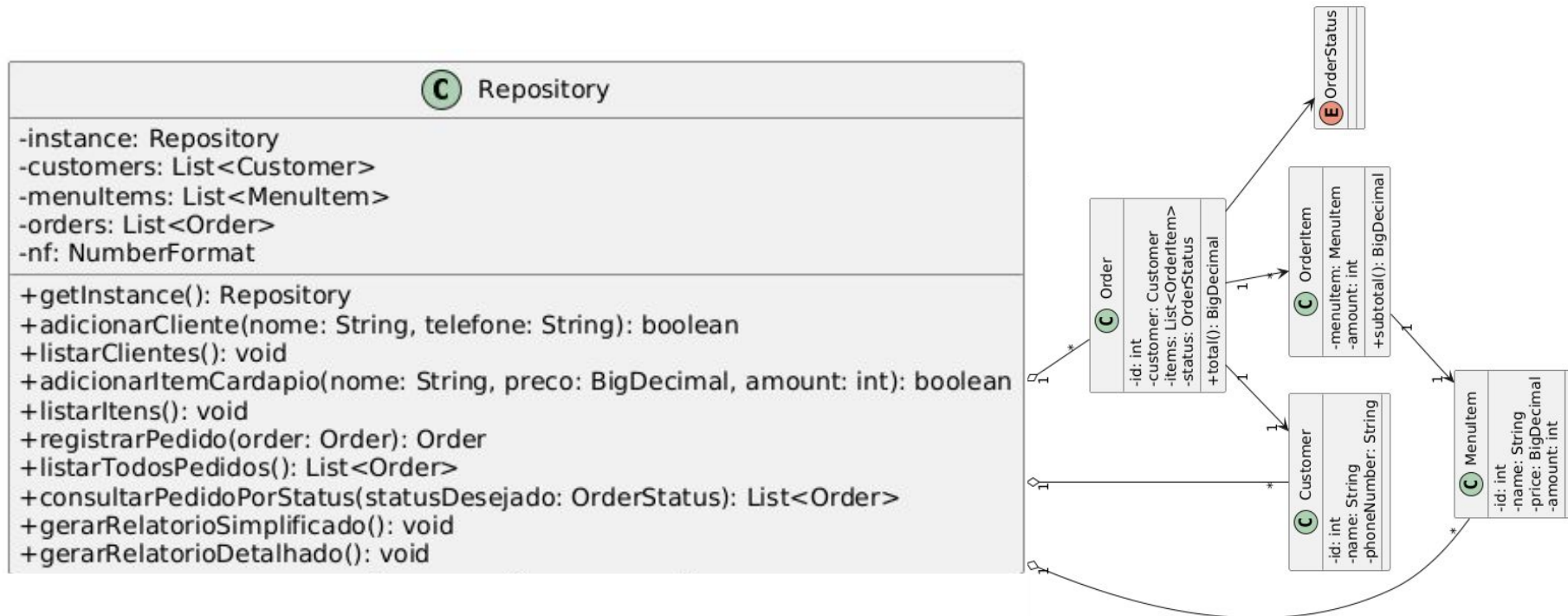
```
// Mantemos a classe 'final' para evitar que alguém tente estender e quebrar noss
public final class Repository { 3 usages  @ Luan Vinicius +1

    // Aqui guardamos a nossa única instância do Repository.
    private static volatile Repository instance; 3 usages

    // --- Nossas listas internas que armazenam os dados do sistema ---
    private final List<Customer> customers; 6 usages
    private final List<MenuItem> menuItems; 6 usages
    private final List<Order> orders; 11 usages
    private final NumberFormat nf = NumberFormat.getCurrencyInstance(new Locale(

    // Construtor privado: aqui garantimos que ninguém fora da classe consiga cri
    private Repository() { 1 usage  @ Luan Vinicius
        this.customers = new ArrayList<>();
        this.menuItems = new ArrayList<>();
        this.orders = new ArrayList<>();
        System.out.println("Repository inicializado. Pronto para uso!");
    }
```

Diagrama



Metodologia



- Primeiro, foi feita a **análise do problema**, identificando que cada parte do sistema possuía suas próprias listas de clientes e pedidos, o que poderia gerar inconsistências. Assim, definiu-se a necessidade de uma **central única de dados**.
- Em seguida, iniciou-se a **implementação**: a classe **Repository** foi refatorada para seguir o padrão **Singleton**, com o **construtor privado**, o método **getInstance()** como único ponto de acesso e a classe marcada como **final**, impedindo heranças que pudessem comprometer o padrão.

Por fim, foram realizados **testes completos** no sistema, simulando ações de um usuário real, como cadastro de clientes, criação de pedidos, atualização de status e geração de relatórios, para confirmar que os dados estavam sendo compartilhados corretamente e que tudo continuava funcionando como esperado.

```
// --- Método de acesso global ---  
// Aqui fizemos a lógica do Singleton: qualquer parte do sistema que precisar do Repository vai usar  
public static synchronized Repository getInstance() { no usages @Luan Vinicius  
    if (instance == null) {  
        instance = new Repository();  
    }  
    return instance;  
}
```

Resultados e discussões



O uso do **Singleton** na classe **Repository** tornou o sistema mais **organizado, confiável e centralizado**, garantindo que toda a aplicação utilize uma **única fonte de dados**.

No entanto, ainda existem **limitações**: as informações são armazenadas apenas em **memória**, sendo perdidas ao encerrar o programa, e as **validações de dados** ainda são simples, podendo ser aprimoradas futuramente.

Vantagens:

- Os dados são centralizados em uma única instância, o que evita inconsistências.
- A classe possui métodos claros para adicionar, listar e consultar informações.
- Os relatórios simples e detalhados ficaram prontos para o uso.

Desvantagens: A lógica de validação de dados ainda é básica (por exemplo, não impede o cadastro de nomes vazios). Como tudo é mantido em memória, os dados não persistem se o sistema for fechado.

Considerações finais



- Transformar o **Repository** em um **Singleton** foi essencial para o sucesso do projeto, pois permitiu **centralizar os dados** e tornar o código mais **organizado e fácil de manter**. A experiência mostrou na prática como um padrão de projeto pode resolver problemas reais de desenvolvimento.
- O próximo passo é **armazenar as informações em um banco de dados**, evitando a perda de dados ao encerrar o programa.
- O maior desafio foi garantir a **segurança na criação da instância**, especialmente em ambientes com múltiplas execuções. Para isso, foi utilizado o **synchronized no método getInstance()**, garantindo um comportamento **estável e robusto** mesmo em situações complexas.

```
=== TiaLu Delivery ===
1 - Adicionar cliente
2 - Listar clientes
3 - Adicionar item ao cardápio
4 - Listar itens do cardápio
5 - Registrar pedido
6 - Consultar pedidos por status
7 - Listar todos os pedidos
8 - Gerar relatório simples
9 - Gerar relatório detalhado
0 - Sair

Escolha uma opção: 3
Nome do item: COCA COLA 3L
Preço do item: 15
Quantidade em estoque: 2
Item 'COCA COLA 3L' adicionado ao cardápio!
```